

Introductions and Meeting Logistics

- Agnibha Sarkar from Turing joined as co-teacher
 - Recent Turing engineer (1.5 years), worked on research aspects of agentic AI
 - Previously managed by Irene, now on platform team running agentic evaluations
 - Background: Electronics and telecommunications from India, multiple ML/NLP internships
 - Contact: agnibha.sarkar@turing.com
- UCSB ACM team introductions completed
 - Mix of CS, Statistics & Data Science majors (1st-3rd year)
 - Interests span LLM security, reinforcement learning, deep learning applications, agentic AI
- Schedule change: Moving from Thursday 8-9am to Friday 8-9am for better availability
- Break planned: Next week meeting, then 2-week break for finals and spring break

Agentic AI Fundamentals and CBRE Case Study

- Agnibha prototyped CBRE voice agent in 4 days under tight client deadline
 - Voice agent for maintenance call routing and classification
 - Successfully demoed to client, now being showcased as example
- Core agentic components demonstrated through CBRE system:
 - Planning: Define objective and steps for complex goals
 - Tool use: API calls, database queries, code execution
 - Memory: Carry context across multiple steps vs one-shot calls
 - Reflection: Self-critique and improvement mechanisms
- One-shot vs agentic approach comparison
 - One-shot: Direct transcript classification with high variation and hallucination
 - Agentic: Extract → Look up → Classify → Check confidence → Route
 - Agentic approach provides consistent routing decisions and better accuracy

Degrees of Autonomy Spectrum

- Low autonomy: Predetermined steps, hardcoded tools, fixed pipeline
 - Example: Gemini internet search with predefined Google search tool
- Semi-autonomous: Agents choose which predefined tools to use
 - CBRE system fits here - has decision-making freedom within defined parameters
 - Can handle branching logic (high vs low priority routing)
 - Iterative entity collection until sufficient information gathered
- High autonomy: Independent decisions, minimal human guidance
 - Creates new tools, adapts dynamically
 - Used for research tasks where humans don't know optimal approach

Task Decomposition Strategy

- Critical decision framework for each workflow step:
 - LLM: Language understanding, entity extraction, ambiguous conversations
 - Tools: Database queries, API calls, vector searches, web hooks
 - Code: Simple deterministic thresholds, confidence scoring, routing logic
- CBRE implementation examples:
 - LLM: Extract problem, location, severity from user transcript
 - Tools: Query historical database, find nearest maintenance centers
 - Code: Assign confidence scores, route based on thresholds
- Edge case handling: Route to human when user provides insufficient context after multiple attempts

Four Core Design Patterns

- Reflection: Agent reviews own output and improves iteratively
 - Self-reflection: Same LLM critiques its work
 - External feedback: Human-in-the-loop or API validation (more powerful)
 - Example: Code debugging where error messages fed back for correction
- Tool use: Specialized functions for specific tasks
 - Database queries, API integrations, code execution
 - Modularity allows swapping tools based on performance
- Planning: Breaking complex goals into manageable steps
 - Planner modes in coding tools (Cursor, Claude) outline before execution
- Multi-agent: Specialized agents collaborate with defined roles
 - Data flows between agents, each optimized for specific function
 - Easier debugging and evaluation of individual components

Industry Context and Technical Insights

- 98% of companies use off-the-shelf models rather than building from scratch
- Agentic AI more relevant skill than traditional fine-tuning
- Fine-tuning deprioritized at Turing (and industry-wide) due to poor ROI
 - Only frontier labs with massive capital continue traditional fine-tuning
 - Turing provides SFT and RLHF data to frontier labs instead
- Model selection strategy: Use best model for each task
 - Gemini excels at vision tasks
 - Claude superior for calculations and reasoning
- Performance advantage: Worse models with agentic patterns can outperform better models with one-shot approaches
- Parallelism and modularity enable faster throughput and easier scaling

Next Steps and Resources

- Python Jupyter notebook required (everything else optional)
 - Sample transcripts and implementation examples provided

- Reflection agent demonstration included
 - API key needed (free tier sufficient for exercises)
 - Food for thought questions to discuss next meeting
 - Framework exploration encouraged: LangChain, LangGraph, Autogen
 - Turing platform details: <https://turing.com>, Agnibha's work: <https://agnibha.sa>
 - Meeting cadence: Next week confirmed, then resume after spring break
-